

Real life examples of Method Overloading :

Example 1:

unlockPhone(PIN pin)

unlockPhone(Pattern pattern)

unlockPhone(Password password)

Example 2:

makePayment(UPI upi)

makePayment(DebitCard card)

makePayment(CreditCard card)

//Programs on method overloading :

WAP to overload a method which describes that while overloading we can change the return type & access modifier

MOL2.java

```
-----
void main()
{
    var result = sum(2.2, 5.6);
    IO.println("Sum of two double is :"+result);

    result = sum(10,20,30);
    IO.println("Sum of three integer is :"+result);

    result = sum(100,200);
    IO.println("Sum of two integer is :"+result);
}

public int sum(int x, int y)
{
    return x + y;
}

protected int sum(int x, int y, int z)
{
    return x+y+z;
}

private double sum(double x, double y)
{
    return x+y;
}
```

MOL3.java

WAP to overload a method with different payment system where the debit card must be of 16 digits

```
void main()
{
    makePayment(12000);
    IO.println(".....");
    makePayment("Scott", 15000);
    IO.println(".....");
    makePayment("John", 18000,"3456987612346526");
}

public void makePayment(double amount)
{
    IO.println("Your Payment amount is :"+amount);
}

public void makePayment(String name, double amount)
{
    IO.println("Your Name is :"+name);
    IO.println("And your Payment amount is :"+amount);
}

public void makePayment(String name, double amount, String debitCard)
{
    if(debitCard.length()!=16)
    {
        IO.println("Invalid Card. It must be 16 digits");
        return;
    }

    IO.println("Your Name is :"+name);
    IO.println("And your Payment amount is :"+amount);
    debitCard = "**** *"+debitCard.substring(12);
    IO.println("Your debit card number is :"+debitCard);
}
```

MOL4.java

WAP to show type promotion applies automatically while overloading a method.

Type Promotion :

Case 1 : 'A' -> char -> int -> long -> float -> double
Case 2 : 12 -> int -> long -> float -> double
Case 3 : 12 -> byte [Compilation error, Narrowing is not supported]
Case 4 : 12 -> char [Compilation error, Narrowing is not supported]
Case 5 : 12 -> short [Compilation error, Narrowing is not supported]
Case 6 : 9L -> int [Compilation error, Narrowing is not supported]
Case 7 : 9L -> long -> float -> double
Case 8 : true -> boolean

```
void main()
{
    accept('A');
}

void accept(char ch)
{
    IO.println("CHAR");
}

void accept(int x)
{
    IO.println("INT");
}

void accept(long x)
{
    IO.println("LONG");
}

void accept(float x)
{
    IO.println("FLOAT");
}

void accept(double x)
{
    IO.println("DOUBLE");
}
```

MOL5.java

```
void main()
{
    accept(9L);
}

void accept(long x)
{
    IO.println("Long");
}

void accept(float x)
{
    IO.println("Float");
}

void accept(double x)
{
    IO.println("Double");
}
```

MOL6.java

WAP to show that if we only change the return type of the method then It is not a valid overload method

```
void main()
{
}

long accept(long x)
{
    return 0;
}

double accept(long x)
{
    return 0;
}
```

Error

Var args in Java :

* It was introduced from JDK 1.5v onwards.

* It is represented by exactly ... **(3 dots)**

* A var args parameter variable can accept **0 to 'n' number of arguments** of same type OR different type also by using Object class.

* Actually, Internally It is an **array variable** which we can use only for **method OR constructor parameter**.